

Import Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV, PredefinedSplit

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully")
```

Libraries imported successfully

Upload Dataset

```
from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

df.head()
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Invistico_Airline.csv to Invistico_Airline.csv

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	...	Online support	Ease of Online booking	s
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0		0	0	2	...	2	3
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0		0	0	3	...	2	3
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0		0	0	3	...	2	2
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0		0	0	3	...	3	1
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0		0	0	3	...	4	2

5 rows × 22 columns

Explore Dataset

```
print("Dataset Shape:")
print(df.shape)

print("\nColumns:")
print(df.columns)
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

Dataset Shape:
(129880, 22)

Columns:

```
Index(['satisfaction', 'Customer Type', 'Age', 'Type of Travel', 'Class',
      'Flight Distance', 'Seat comfort', 'Departure/Arrival time convenient',
      'Food and drink', 'Gate location', 'Inflight wifi service',
      'Inflight entertainment', 'Online support', 'Ease of Online booking',
      'On-board service', 'Leg room service', 'Baggage handling',
      'Checkin service', 'Cleanliness', 'Online boarding',
      'Departure Delay in Minutes', 'Arrival Delay in Minutes'],
      dtype='object')
```

```
Missing Values:
satisfaction                0
Customer Type                0
Age                          0
Type of Travel              0
Class                       0
Flight Distance              0
Seat comfort                 0
Departure/Arrival time convenient 0
Food and drink              0
Gate location                0
Inflight wifi service        0
Inflight entertainment        0
Online support                0
Ease of Online booking        0
On-board service              0
Leg room service              0
Baggage handling              0
Checkin service               0
Cleanliness                   0
Online boarding               0
Departure Delay in Minutes    0
Arrival Delay in Minutes     393
dtype: int64
```

Data Cleaning

```
# Remove missing values
```

```
df = df.dropna()
```

```
print("After cleaning:")
print(df.shape)
```

After cleaning:
(129487, 22)

Identify Target Variable

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 129487 entries, 0 to 129879
Data columns (total 22 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   satisfaction         129487 non-null object
 1   Customer Type        129487 non-null object
 2   Age                  129487 non-null int64
 3   Type of Travel       129487 non-null object
 4   Class                129487 non-null object
 5   Flight Distance      129487 non-null int64
 6   Seat comfort         129487 non-null int64
 7   Departure/Arrival time convenient 129487 non-null int64
 8   Food and drink       129487 non-null int64
 9   Gate location        129487 non-null int64
10   Inflight wifi service 129487 non-null int64
11   Inflight entertainment 129487 non-null int64
12   Online support        129487 non-null int64
13   Ease of Online booking 129487 non-null int64
```

```
14 On-board service          129487 non-null int64
15 Leg room service          129487 non-null int64
16 Baggage handling          129487 non-null int64
17 Checkin service           129487 non-null int64
18 Cleanliness                129487 non-null int64
19 Online boarding            129487 non-null int64
20 Departure Delay in Minutes 129487 non-null int64
21 Arrival Delay in Minutes   129487 non-null float64
dtypes: float64(1), int64(17), object(4)
memory usage: 22.7+ MB
```

Encode Categorical Variables

```
# Separate features and target
target = 'satisfaction'
X = df.drop(target, axis=1)
y = df[target]

# Encode categorical features

categorical_columns = X.select_dtypes(
    include=['object']
).columns

encoder = LabelEncoder()

for col in categorical_columns:

    X[col] = encoder.fit_transform(
        X[col]
    )

# Encode target

y = encoder.fit_transform(y)

X.head()
```

	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	Inflight wifi service	...	Online support	Ease of Online booking	On-board service
0	0	65	1	1	265	0	0	0	2	2	...	2	3	3
1	0	47	1	0	2464	0	0	0	3	0	...	2	3	4
2	0	15	1	1	2138	0	0	0	3	2	...	2	2	3
3	0	60	1	1	623	0	0	0	3	3	...	3	1	1
4	0	70	1	1	354	0	0	0	3	4	...	4	2	2

5 rows × 15 columns

Three-Way Data Split

```
# Split training + remaining

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Split validation and test

X_val, X_test, y_val, y_test = train_test_split(
```

```

X_temp,
y_temp,
test_size=0.50,
random_state=42,
stratify=y_temp
)

print("Training:", X_train.shape)

print("Validation:", X_val.shape)

print("Testing:", X_test.shape)

```

```

Training: (90640, 21)
Validation: (19423, 21)
Testing: (19424, 21)

```

Combine Train and Validation for PredefinedSplit

```

X_train_val = pd.concat(
    [X_train, X_val]
)

y_train_val = np.concatenate(
    [y_train, y_val]
)

# -1 means training
# 0 means validation

validation_fold = np.concatenate(
    [
        np.full(len(X_train), -1),
        np.zeros(len(X_val))
    ]
)

ps = PredefinedSplit(
    test_fold=validation_fold
)

print(ps)

PredefinedSplit(test_fold=array([-1, -1, ..., 0, 0]))

```

Random Forest GridSearch Hyperparameter Tuning

```

rf = RandomForestClassifier(
    random_state=42
)

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 5, 10],
    "min_samples_split": [2, 5]
}

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,

```

```

cv=ps,

scoring="f1",

n_jobs=-1

)

```

```

grid_search.fit(
    X_train_val,
    y_train_val
)

```

```
print("Best Parameters:")
```

```
print(grid_search.best_params_)
```

```

Best Parameters:
{'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}

```

Best Random Forest Model

```
best_rf = grid_search.best_estimator_
```

```

rf_prediction = best_rf.predict(
    X_test
)

```

```

print(
    classification_report(
        y_test,
        rf_prediction
    )
)

```

	precision	recall	f1-score	support
0	0.94	0.96	0.95	8791
1	0.97	0.95	0.96	10633
accuracy			0.95	19424
macro avg	0.95	0.95	0.95	19424
weighted avg	0.95	0.95	0.95	19424

Performance Metrics

```

rf_accuracy = accuracy_score(
    y_test,
    rf_prediction
)

```

```

rf_precision = precision_score(
    y_test,
    rf_prediction,
    average="weighted"
)

```

```

rf_recall = recall_score(
    y_test,
    rf_prediction,
    average="weighted"
)

```

```

rf_f1 = f1_score(
    y_test,
    rf_prediction,

```

```

        average="weighted"
    )

    rf_results = pd.DataFrame({

        "Model":["Random Forest"],

        "Accuracy":[rf_accuracy],

        "Precision":[rf_precision],

        "Recall":[rf_recall],

        "F1 Score":[rf_f1]

    })

    rf_results

```

	Model	Accuracy	Precision	Recall	F1 Score
0	Random Forest	0.954489	0.954701	0.954489	0.954523

Decision Tree Comparison

```

dt = DecisionTreeClassifier(
    random_state=42
)

dt.fit(
    X_train,
    y_train
)

dt_prediction = dt.predict(
    X_test
)

dt_results = pd.DataFrame({

    "Model":["Decision Tree"],

    "Accuracy":[
        accuracy_score(y_test,dt_prediction)
    ],

    "Precision":[
        precision_score(
            y_test,
            dt_prediction,
            average="weighted"
        )
    ],

    "Recall":[
        recall_score(
            y_test,
            dt_prediction,
            average="weighted"
        )
    ],

    "F1 Score":[
        f1_score(
            y_test,
            dt_prediction,
            average="weighted"
        )
    ]
})

```

```
]
})

dt_results
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	0.932043	0.932022	0.932043	0.932028

Final Comparison

```
comparison = pd.concat(
    [
        rf_results,
        dt_results
    ]
)
```

```
comparison
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Random Forest	0.954489	0.954701	0.954489	0.954523
0	Decision Tree	0.932043	0.932022	0.932043	0.932028

Interpretation

```
print("""
Interpretation:

Random Forest combines multiple decision trees,
which usually improves prediction stability and reduces
overfitting compared with a single Decision Tree.

The F1-score was used because it balances precision
and recall, making it useful when classification errors
need to be considered.

The GridSearchCV process tested multiple combinations
of Random Forest parameters and selected the best model
based on validation performance.

Limitations:
- Model performance depends on feature quality.
- Random Forest requires more computational resources.
- Dataset imbalance may affect evaluation.

Future improvements:
- Apply feature importance analysis.
- Try other algorithms such as XGBoost.
- Perform cross-validation.
""")
```

```
Interpretation:

Random Forest combines multiple decision trees,
which usually improves prediction stability and reduces
overfitting compared with a single Decision Tree.

The F1-score was used because it balances precision
and recall, making it useful when classification errors
need to be considered.

The GridSearchCV process tested multiple combinations
of Random Forest parameters and selected the best model
based on validation performance.

Limitations:
- Model performance depends on feature quality.
- Random Forest requires more computational resources.
```

- Dataset imbalance may affect evaluation.

Future improvements:

- Apply feature importance analysis.
- Try other algorithms such as XGBoost.
- Perform cross-validation.